



Name:- JayKishan J Saharan,

Div:- D Roll no:- 26

Course/Program:- BTech CSE SY

Subject:- DSA

DSA Theory Assignment 3.

15/10/24

Q.1) What is a stack? Describe stack as an ADT.
What is a Queue? Describe Queue as an ADT.

Sol:-

Stack

- A stack is a linear data structure that follows the last-in, first-out (LIFO) principle, meaning the last element added is the first one to be removed. It can be visualized as a pile of plates, where new plates are added to the top and plates are removed from the top.

As an Abstract Data type (ADT), a stack is defined by its behavior and the operations it supports, rather than its specific implementation details. The core operations of a stack ADT are:

- 1) Push(element): Adds an element to the top of stack.
- 2) Pop(): Removes and Returns the elements at the top of the stack.
- 3) Peek()/TOP(): Returns the elements at the top of the stack without removing it.
- 4) isEmpty(): checks if the stack is empty. Returns true if empty, false otherwise.
- 5) Size(): Returns the number of elements in the stack.



Queue

- A queue is an abstract data type (ADT) that models a real-world line, following the First-In, first-out (FIFO) principle where the first element added is the first one removed.

Queue also uses a set of operations like enqueue(), dequeue(), peek(), isfull(), isempty(), without specifying how it is implemented and thus queue is an abstract data type.

Q.3) What is recursion? Explain how a stack is used in recursion particularly in computing the factorial of a number.

Solⁿ: Recursion is a technique in programming where a function calls itself to solve a problem.

- when calculating factorial, stacks can be used with recursion to find out factorial.
- Let's assume that we are calculating factorial of 5, then function will be fact(5) and stack = [].

⇒ Ex:- fact(5) is called → Push on stack ———

n-1 → fact(4) is called → Push on stack ———	fact(1)
n-1 → fact(3) is called → Push on stack ———	fact(2)
n-1 → fact(2) is called → Push on stack ———	fact(3)
n-1 → fact(1) is called → Push on stack ———	fact(4)
	fact(5)

(stack).

- when repeated calling of function, it gets pushed onto the call stack and when fact(1) is called and it's a base case and it returns a value (1) then pop happens.



⇒

fact(1)	→ return 1
fact(2)	→ Pop out → returns $1 \times 2 = 2$
fact(3)	→ Pop out → returns $2 \times 3 = 6$
fact(4)	→ Pop out → returns $6 \times 4 = 24$
fact(5)	→ Pop out → returns $24 \times 5 = 120$

∴ fact(5) is 120 with the help of stack and recursion.

Q.4) Write Pseudocode to implement push and pop operations on a stack using arrays.

Solⁿ:-

```
function Push (value)
    if (isfull() == True)
        cout "stack overflow, cannot push"
    else
        Increment top by 1 (top = top + 1)
        Set arr[top] = value
        cout "pushed onto the stack"
// End of function.
```

→

```
function Pop ()
    if (isempty() == True)
        cout "stack underflow, cannot pop"
    else
        cout arr[top], "popped from stack"
        Decrement top by 1 (top = top - 1)
```

Q.5) Explain step by step how a stack is used to convert infix expression $A * (B + C) * D$ into postfix.

Solⁿ:-



	a		a	
	*		a	Stack
	(		a	*
Input:	b	output	ab	)
	+		ab	+
	c		abc	(
	)		abc+	*
	*		abc+*	
	d		abc++d*	

Postfix $\Rightarrow$ $abc++d*$

Q.6) Write an algorithm to convert a given infix to its equivalent postfix expression.

Solⁿ:- Algorithm:

- Step: 1) Scan the input infix expression one by one from left to right.
- Step: 2) If the input symbol is an operand then append it to the postfix expression.
- Step: 3) If input symbol is '(' then push it onto the stack.
- Step: 4) If input symbol is ')' then pop all contents till it matches '(' then append its contents to the output postfix expression.
- Step: 5) If input symbol is an operator (+, -, /, *) then push it onto the stack. However first we need to pop operators which are already on the stack with higher or equal precedence and append to output.
- Step: 6) If input is over, then pop all the remaining symbols from stack and append them to the output.



Q.7) write Pseudo code to implement a Simple queue and circular queue including enqueue(), dequeue

solⁿ:- Simple Queue:-

Initialize: front = -1

rear = -1

Size = 5

arr = array of size 5

isEmpty():

function isEmpty():

Return (front == -1) or (front > rear).

isFull():

function isFull():

Return (rear == size - 1)

enqueue (value):

function enqueue (value):

if isFull() is True:

Print "Queue overflow"

Return

if isEmpty() is True:

front = 0

rear = 0

else:

rear = rear + 1

arr[rear] = value

Print "Enqueued", value.



dequeue():

function dequeue():

if isempty() is True:

Print "Queue Underflow"

Return

value = arr[front]

if front == rear:

front = -1

rear = -1

else:

front = front + 1

Print "Element deleted", value.

* Circular Queue:-

Initialize: front = -1

rear = -1

Size = 5

arr = array of size 5

function isempty():

Return (front == -1)

function isfull():

Return ((rear+1) mod size) == front

function Enqueue(value):

if isfull() is True:

Print "Queue Overflow"

if isempty() is True:



front = 0

rear = 0

else:

rear = (rear + 1) mod Size

arr[rear] = Value

Print "Enqueued", value.

function dequeue():

if isempty() is True:

Print "Queue Underflow"

Return

value = arr[front]

if front == rear:

front = -1

rear = -1

else:

front = (front + 1) mod Size

Print "Deleted element", value.

function display():

if isempty() is True:

Print "Queue is empty"

Return

Print "Element in Queue are:"

Set i = front

Loop indefinitely:

Print arr[i]

if i == rear:

Break the loop

i = (i + 1) mod Size



Q.8) Evaluate the advantages, disadvantages of using a circular queue over simple queue. Analyse scenarios where one will be preferred over the other.

Sol:-

Advantages of circular over simple queue:-

1) Efficient use of space:

Even if there are empty places at the front due to dequeued elements, circular queue reuses those free spaces by wrapping around to beginning of array.

2) No wasted memory:

Circular queue eliminates wasted space problem of a simple queue by making array behave like it's connected end to start forming a circle.

Disadvantages of circular queue:

1) Complexity of Implementation:

Logic for a circular queue is more complex and error-prone.

2) Fixed size (Static):

Standard array based circular queue has a fixed maximum capacity and the problem is more for circular. → circular queue are ideal for situations with fixed memory constraints and a continuous flow of data where old data becomes irrelevant. (Ex - task scheduling)

→ Simple queue are ideal for situations when the size is unpredictable or you need straight forward FIFO scheduling / processing.



9.9). Compare Simple queue, circular, double-ended queue and priority queue based on their operations and typical applications.

Solⁿ:-

	Type	Insertion	Deletion	Storage	Application
1)	Simple Linear Queue	At the rear end only.	From the front end only.	wastes space when dequeued	Request Handling.
2)	Circular queue.	At the rear (wraps end is reached)	From the front (also wraps around)	No memory wasted	CPU Scheduling
3)	Double-Ended Queue	Insertion at both front and rear end.	Deletion at both ends	a bit more complexity than Simple queue.	Undo, Redo operations.
4)	Priority Queue.	Inserted anywhere but based on priority	from the highest priority element.	Different when heap array is used	Job Scheduling in OS.

h